Case 02 · מסמך עיצוב: Fire-Sale Cascade Simulator

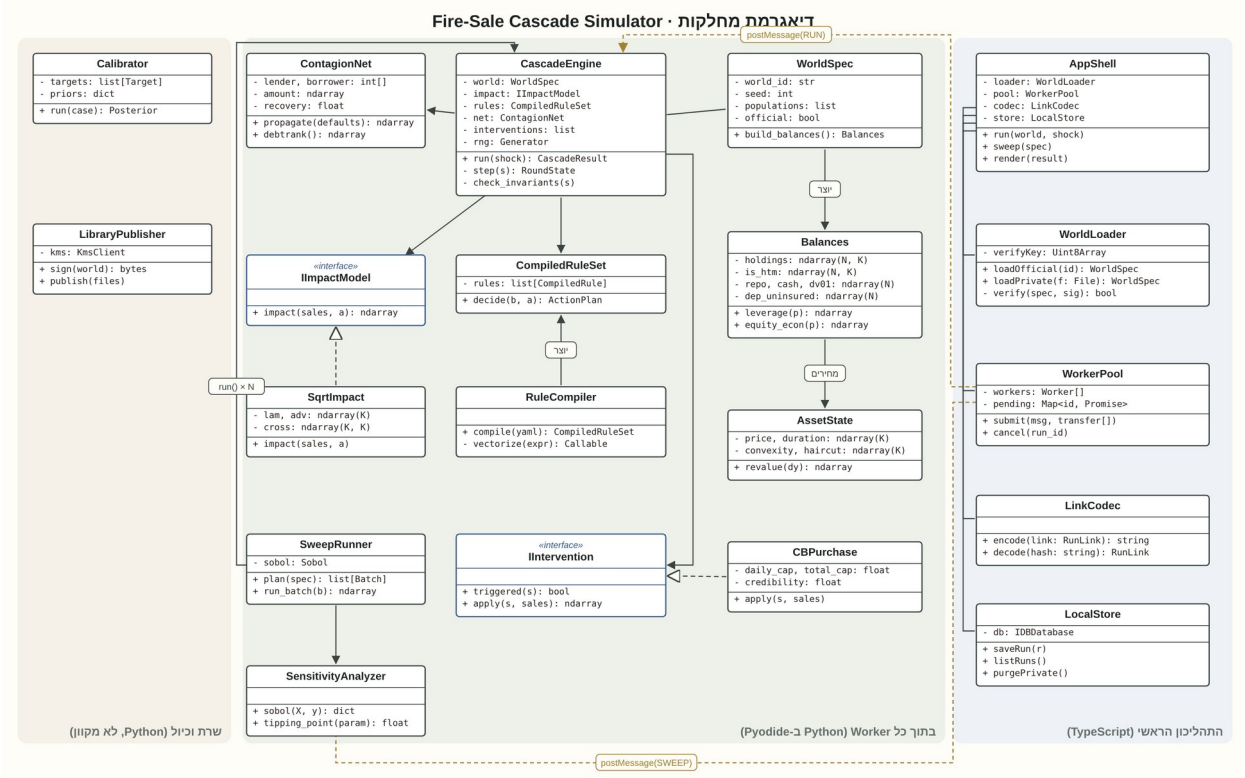
השלב שאחרי הארכיטקטורה. הארכיטקטורה קבעה אילו רכיבים יש ומה עובר ביניהם. העיצוב יורד רמה אחת: אילו מחלקות יש בתוך כל רכיב, באילו מצבים כל אובייקט חי, מי מדבר עם מי ובאיזה סדר, אילו תהליכונים רצים במקביל, ואיך נראות הטבלאות. המבנה זהה למסמך העיצוב של Case 01 ולמסמך של שרת הטריוויה.

עקרונות עיצוב

ארבעה עקרונות, ישירות מהדפוסים שנבחרו בארכיטקטורה:

- **המנוע טהור:** CascadeEngine.run() מקבל עולם, זעזוע ו-seed ומחזיר תוצאה. אין בו קריאה לרשת, לשעון, ל-IndexedDB או ל-DOM. כל מה שנוגע בעולם החיצון נמצא בתהליכון הראשי (Functional Core, Imperative Shell).
- **סוכן הוא אינדקס, לא אובייקט:** אין מחלקה Agent. יש Balances שבה כל תכונה היא מערך של כל הסוכנים, וכל שלב בסבב הוא פעולה וקטורית על המערכים (Data-Oriented Design).
- **ממשק לכל דבר שמשתנה בין מחקרים:** מודל השפעת המחיר וההתערבות הם ממשקים (IImpactModel, IIntervention), כי בדיוק אותם חוקרים ירצו להחליף. הכללים הם נתונים ולא קוד (Interpreter).
- **אין תוצאה בלי טווח:** הרצה בודדת לא מוצגת לבד. התהליכון הראשי שולח אחריה סריקה ברירת מחדל, ורק כשהיא חוזרת מוצגת התוצאה עם p10, p50 ו-p90.

דיאגרמת מחלקות



שלושה אזורים, ושלוש שפות ריצה. מימין, התהליכון הראשי של הדפדפן ב-TypeScript: הוא מנהל ממשק, טעינה ושמירה, ולא מחשב. באמצע, מה שרץ בתוך כל Worker: פייתון בתוך Pyodide, עם NumPy. משמאל, מה שרץ בשרת ולא אצל המשתמש: כיול וחתימה.

הקווים המקווקווים הם postMessage: אין קריאה ישירה בין התהליכון הראשי למנוע. זה גם מה שמאפשר להריץ את אותו מנוע בדיוק באשכול הכיול, בלי דפדפן בכלל.

תפקיד כל מחלקה

מחלקה	אזור	תפקיד	מה היא לא עושה
AppShell	ראשי	מקבלת קלט, מפעילה הרצה וסריקה, מציירת	לא מחשבת שום דבר מהמודל
WorldLoader	ראשי	טוענת עולם מהספרייה או מקובץ פרטי, בודקת סכמה וחתימת Ed25519	לא שולחת קובץ פרטי לשום מקום
WorkerPool	ראשי	מחזיקה את ה-Workers, מחלקת עבודה ומאחדת תוצאות	לא יודעת מה בתוך ההודעה
LinkCodec	ראשי	מקודדת הגדרת הרצה ל-fragment בכתובת ובחזרה	לא מקודדת מאזנים, רק הגדרה
LocalStore	ראשי	שומרת הרצות, סריקות ועולמות פרטיים ב-IndexedDB	לא מסנכרנת לשרת
CascadeEngine	Worker	לולאת הסבבים: שערוך, החלטה, סליקה, התערבות, השפעה, הדבקה	לא יודעת מה זה "קרן LDI"
WorldSpec	Worker	תיאור העולם: אוכלוסיות, פרמטרים, seed, האם רשמי	
Balances	Worker	מאזני כל הסוכנים כמערכים (N סוכנים, K נכסים)	אין לה לוגיקת החלטה
AssetState	Worker	מחיר, משך, קמירות ותספורת לכל נכס; שערוך לפי שינוי תשואה	
IImpactModel	Worker	ממשק: מכירות נטו נכנסות, שינוי מחיר יוצא	
LinearImpact, SqrtImpact, DealerCapacity	Worker	שלושת המימושים: Kyle לינארי, שורש ריבועי, ומגבלת מאזן של עושי השוק	
RuleCompiler	Worker	מתרגמת YAML לפונקציות וקטוריות, פעם אחת לכל הרצה	לא מריצה eval על קלט
CompiledRuleSet	Worker	מחזירה ActionPlan: כמה כל סוכן מוכר, מעמיד או מושך	
IIntervention	Worker	ממשק: triggered(state) ו-apply(state, sales)	
CBPurchase, CollateralRelief, DepositGuarantee, ParFacility	Worker	ארבעת הכלים מהאפיון: רכישות בנק מרכזי, הקלה בבטוחות, ביטוח פיקדונות, מסגרת לפי ערך נקוב	
ContagionNet	Worker	גרף חובות ישירים; הפסד מחדל עובר למלווים; DebtRank מראש	
SweepRunner	Worker	מריצה מקטע של רצף Sobol ומחזירה מערך תוצאות	

מחלקה	אזור	תפקיד	מה היא לא עושה
SensitivityAnalyzer	Worker	מדדי Sobol (SALib) ונקודת מפנה לכל פרמטר	
Calibrator	שרת	מול ABC-SMC (pyabc) נתוני יעד של מקרה היסטורי	לא רץ בדפדפן
LibraryPublisher	שרת	חותמת עולמות וכיולים ומפרסמת אותם כקבצים סטטיים	המפתח הפרטי לא יוצא מה-KMS

פרוטוקול ההודעות

כמו בשרת הטריוויה, כל הודעה בין התהליכון הראשי ל-Worker מוגדרת מראש: שדה type, מזהה ריצה, ותוכן. ההבדל: כאן אין רשת, יש postMessage, ומערכים גדולים עוברים כ-Transferable כדי שלא יועתקו.

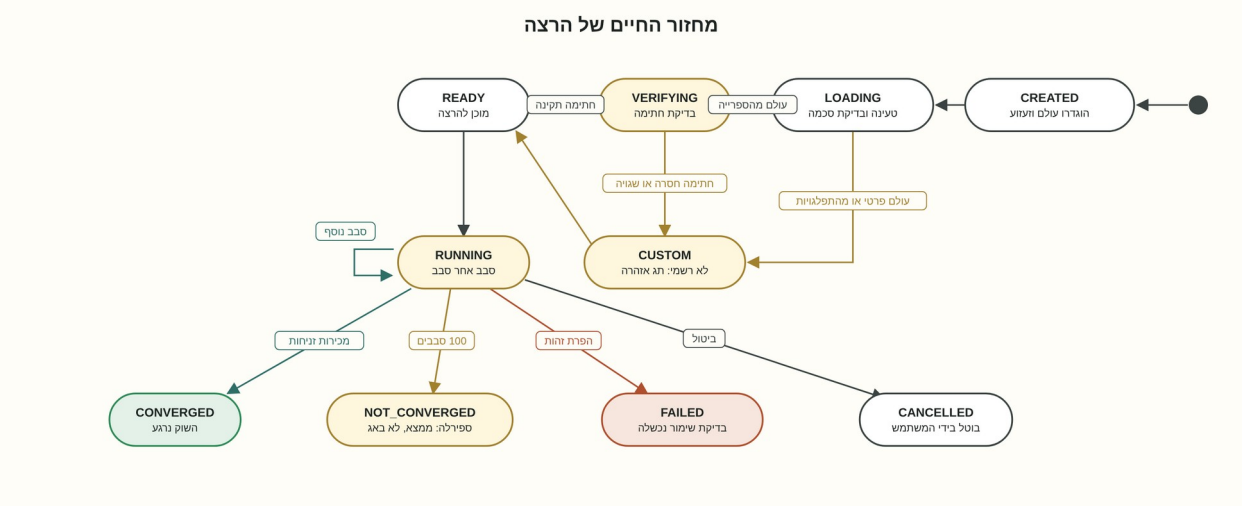
```
→ Worker   { type: "RUN",   run_id, engine_ver, spec: WorldSpec,
             buffers: { holdings, repo, cash, ... }  [transfer],
             shock, interventions[], seed }
→ Worker   { type: "SWEEP", run_id, batch_idx, base_spec, params: Float64Array
[transfer] }
→ Worker   { type: "CANCEL", run_id }
← Worker   { type: "PROGRESS", run_id, done, total }
← Worker   { type: "RESULT",   run_id, rounds: RoundState[] (typed arrays), converged:
bool, summary }
← Worker   { type: "SWEEP_RESULT", run_id, batch_idx, outcomes: Float64Array [transfer] }
← Worker   { type: "ERROR",    run_id, code, detail }
```

קוד שגיאה	מתי	מה מוצג
E_SCHEMA	העולם לא עומד בסכמה (שדה חסר, מספר שלילי)	השדה הבעייתי, ולא מריצים
E_RULES	YAML לא חוקי או משתמש בפעולה שלא קיימת	השורה בקובץ
E_INVARIANT	בדיקת שימור נכשלה: נכסים לא שווים התחייבויות ועוד הון, או מכירה גדולה מהאחזקה	הסבב, הסוכן, והפער. זה באג, לא ממצא
E_RESOURCE	ה-Worker חרג מזיכרון או מזמן	הצעה להקטין את העולם

שימו לב למה שאינו שגיאה: מפל שלא התכנס אחרי 100 סבבים חוזר כ-RESULT עם converged: false. ספירלה שלא נגמרת היא ממצא, והממשק מציג אותה ככזו.

מכונות מצבים

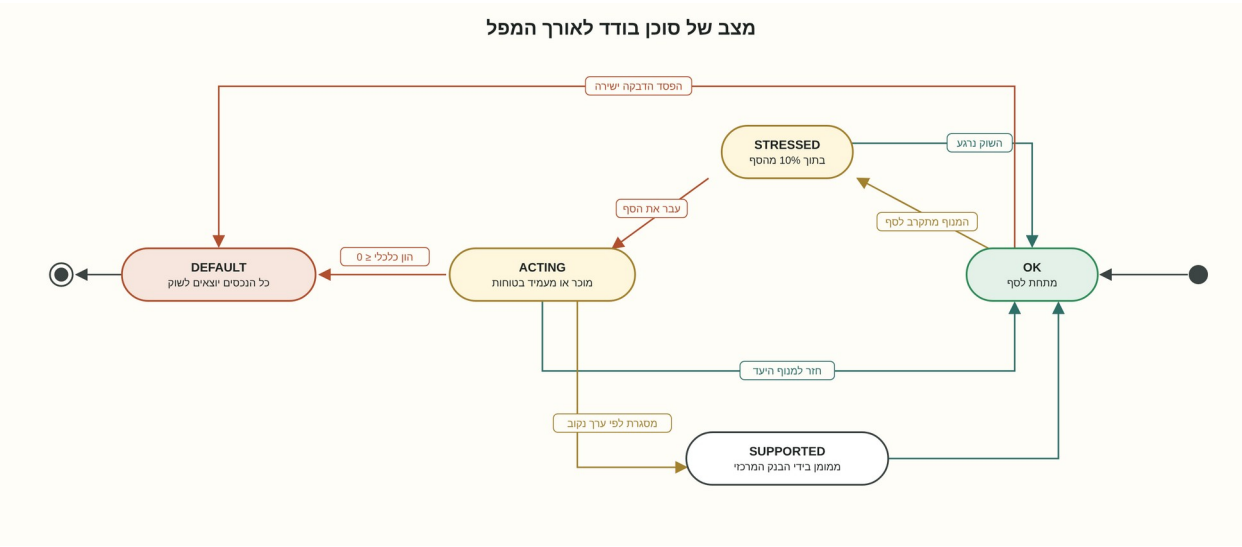
מחזור החיים של הרצה



הרצה היא האובייקט שהמשתמש רואה, וכל מצב שלה מוצג בממשק.

- VERIFYING מתרחש רק לעולם מהספרייה. עולם פרטי או עולם שנבנה מהתפלגויות עובר ישר ל-CUSTOM, ומקבל תג "לא רשמי" שלא ניתן להסיר.
- **חתימה שגויה לא עוצרת את ההרצה. היא מורידה אותה ל-CUSTOM. זו החלטה מכוונת:** המשתמש עדיין יכול לחקור, אבל אף צילום מסך לא יציג את התוצאה כרשמית.
- ארבעה מצבים סופיים, ורק אחד מהם (FAILED) הוא תקלה. NOT_CONVERGED מוצג כתוצאה לכל דבר, בהדגשה.

מצב של סוכן בודד

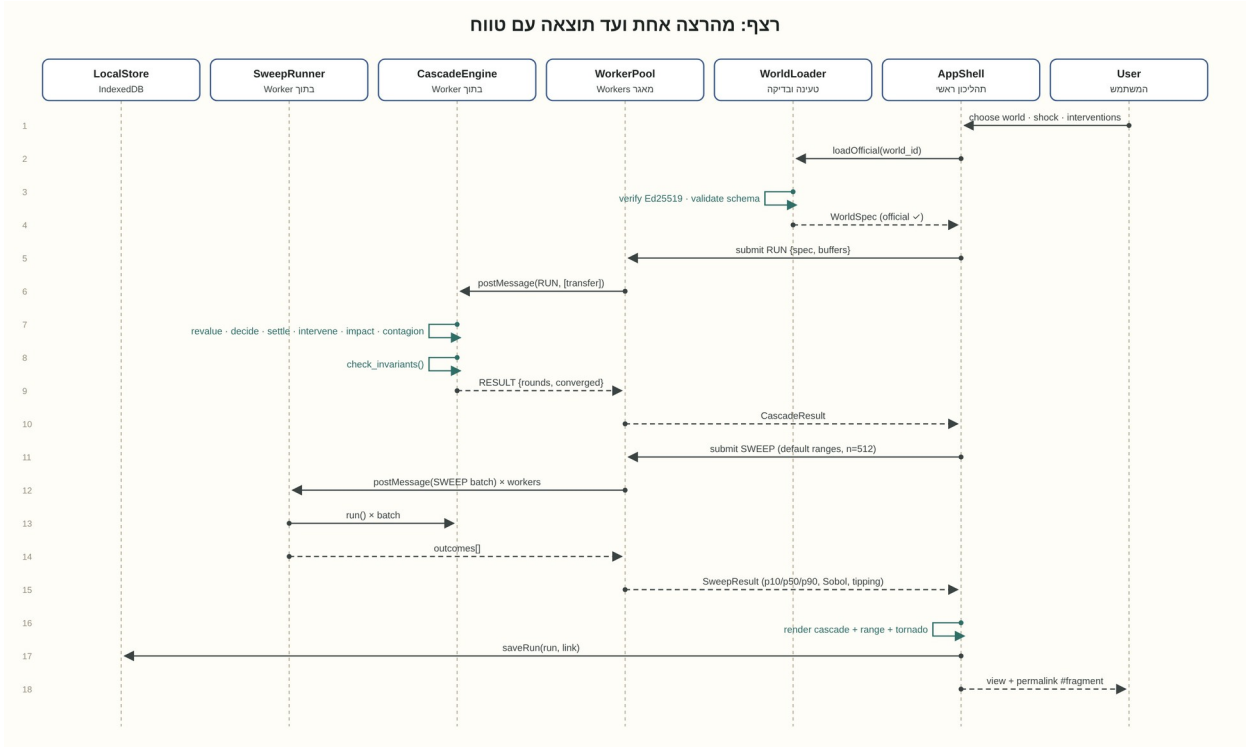


המכונה הזו לא קיימת כאובייקט: היא מחושבת בכל סבב כמערך מצבים, ערך אחד לכל סוכן. היא נועדה לתצוגה ולהבנה: היא מסבירה למשתמש למה סוכן מכר.

- STRESSED הוא מצב אזהרה בלבד, בתוך 10% מהסף. הוא לא משנה התנהגות, אבל הוא מה שמסומן בצהוב בתצוגה, ומראה כמה השוק קרוב לנקודת המפנה.
- **הדרך ל-DEFAULT יכולה לעקוף את כל השאר:** הפסד מהדבקה ישירה (סוכן אחר שפשט רגל ולא החזיר הלוואה) יכול להפיל סוכן שהיה במצב תקין.

- SUPPORTED קיים רק כשמוגדרת מסגרת לפי ערך נקוב. סוכן במצב הזה לא מוכר, ולכן לא מזיז מחיר. זה בדיוק המנגנון שבו התערבות שוברת את המעגל.

דיאגרמת רצף



הרצף מראה פעולה אחת של משתמש, מקצה לקצה. שלוש נקודות:

- **שלבים 5 ו-6:** המאזנים עוברים כ-Transferable. אחרי השליחה, התהליכון הראשי כבר לא יכול לגשת אליהם. זה לא באג, זו ההבטחה שאין שני תהליכונים על אותו זיכרון.
- שלב 11 קורה אוטומטית, בלי שהמשתמש ביקש. זה התרגום לקוד של העיקרון "אין תוצאה בלי טווח".
- **שלב 17:** ההרצה נשמרת מקומית, והקישור הוא fragment (אחרי #). דפדפן לא שולח fragment לשרת, ולכן גם הגדרת ההרצה לא מגיעה לשום מקום שהמשתמש לא שלח אליו בעצמו.

מודל תהליכונים



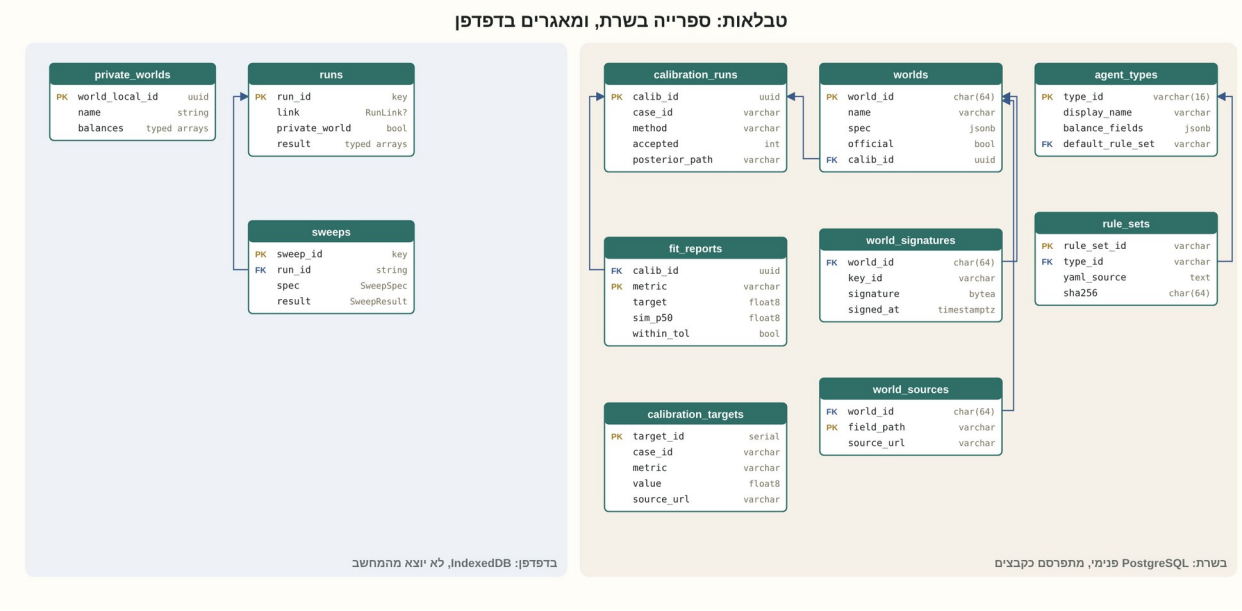
תהליכון	כמה	מה עושה	איך מסתנכרן
Main thread	אחד	ממשק, טעינה, ציור, IndexedDB	לא מחכה אף פעם: הכול async

איך מסתנכרן	מה עושה	כמה	תהליכון
Promise לכל מקטע	מחלק מקטעים, אוסף תוצאות, שולח מחדש מקטע שנפל	חלק מהתהליכון הראשי	WorkerPool
הודעות בלבד	Pyodide, NumPy, המנוע	navigator.hardwareConcurrency פחות 1	Worker 1..N
Dask futures	סבבי ABC-SMC	באשכול הכיול	Dask scheduler + workers

שלושה כללים

- **אין זיכרון משותף, ולכן אין נעילות:** לא SharedArrayBuffer ולא Atomics. כל תהליכון מחזיק את הנתונים שלו, והבעלות עוברת עם ההודעה.
- **seed נגזר מאינדקס המקטע:** seed = hash(base_seed, batch_idx). כך הסריקה נותנת אותה תוצאה גם אם Worker 3 סיים לפני Worker 1, וגם אם Worker קרס והמקטע רץ מחדש במקום אחר.
- **Worker אחד נשאר פנוי:** המאגר משאיר ליבה לתהליכון הראשי, כדי שהממשק לא ייתקע בזמן סריקה.

עיצוב טבלאות



שני מקומות נפרדים, בלי שום קשר ביניהם. בשרת, PostgreSQL פנימי שממנו מייצאים קבצים סטטיים חתומים. בדפדפן, IndexedDB שלא יוצא מהמחשב.

בשרת (פנימי, מתפרסם כקבצים)

טבלה	מה נשמר	הערה
agent_types	חמשת סוגי הסוכנים והשדות במאזן של כל אחד	LDI, בנק, PCF, מבטחת, MMF
rule_sets	קובץ ה-YAML של כל ערכת כללים, עם גיבוב	גרסה חדשה היא שורה חדשה, לא עדכון
worlds	הגדרת עולם, האם רשמי, ואיזה כיול הזין אותו	world_id הוא גיבוב התוכן
world_signatures	חתימה, מזהה מפתח, זמן	עולם בלי חתימה לא מסומן רשמי
world_sources	מקור לכל שדה בעולם	כל מספר בעולם רשמי עם קישור

טבלה	מה נשמר	הערה
calibration_targets	נתוני יעד מהמקרה ההיסטורי (למשל, תנועת התשואה בבריטניה, ספטמבר 2022)	עם מקור
calibration_runs	הרצת כיול: שיטה, מספר מקובלים, נתיב ההתפלגות	
fit_reports	לכל מדד: יעד, חציון מדומה, האם בטווח	מה שלא בטווח מפורסם באדום

בדפדפן (IndexedDB)

מאגר	מה נשמר	הערה
runs	הגדרה, קישור, תוצאה כמערכים טיפוסיים	link ריק לעולם פרטי
sweeps	הגדרת סריקה ותוצאה	שייכת להרצה
private_worlds	מאזנים שהמשתמש העלה	purgePrivate() מוחק הכול בלחיצה

סדר מימוש לפי גרסאות

אותו כלל כמו ב-Case 01: קודם הליבה, ורק אחר כך מה שעוטף אותה. הגרסאות תואמות לספרינטים בארכיטקטורה.

גרסה	מה נבנה	מבחן היציאה
V0.1	CascadeEngine, Balances, AssetState, שני סוגי סוכנים, בדיקות שימור, בפייתון רגיל	משחזר את מספרי ה-POC: 0.6 הופך ל-1.125
V0.2	RuleCompiler עם YAML, שלושת מודלי ההשפעה, חמשת סוגי הסוכנים	אותה ערכת כללים ב-YAML ובקוד נותנת תוצאה זהה
V0.3	Pyodide בתוך Worker, WorldLoader, AppShell בסיסי	אותו seed בדפדפן ובפייתון רגיל: תוצאה זהה עד הביט
V0.4	WorkerPool, SweepRunner, SensitivityAnalyzer, טורנדו	512 עולמות על 8 ליבות בפחות מ-10 שניות; Worker שנהרג באמצע לא משנה תוצאה
V0.5	ContagionNet, DebtRank, ארבע ההתערבויות	התערבות כבויה: זהה ל-V0.4. רכישות בנק מרכזי: המפל נעצר
V1.0	LinkCodec, ספרייה חתומה, כיול, LocalStore	קישור שנפתח במחשב אחר משחזר את אותה הרצה; עולם שחתימתו שונתה מסומן לא רשמי

מה המסמך הזה סוגר, ומה לא

אחרי מסמך העיצוב, המנוע מוגדר עד רמת המחלקה וההודעה, וכל הבטחה מהאפיון (שחזור, טווח, פרטיות של עולם פרטי) מתורגמת למבנה קונקרטי.

ומה שלא נסגר, גם כאן, הוא אותו דבר שבדיקת הסגירה בארכיטקטורה מצאה: בטבלת worlds יש שדה spec, ובתוכו המנוף הממוצע של השוק. בעולם רשמי הוא מגיע מכיול על מקרה היסטורי. לשוק של היום אין לו מקור. העיצוב לא יכול לפתור את זה, כי זו לא בעיית עיצוב.